Proyecto EVRP-Module: estado, algoritmo ACO y experimentos

Resumen de la conversación de trabajo (ACO frente a Guided Local Search de OR-Tools). Documento para compartir con la colaboradora del proyecto. Fecha de exportación: 13 de octubre de 2026.

Cómo leer este documento. Parte de la pregunta «qué tiene implementado actualmente el proyecto y cómo está implementado el ACO» y sigue en orden cronológico lo que se pidió, se cambió y se midió después. Cada tabla de resultados indica el número de semillas y el tiempo usados. Las secciones 1 y 2 describen el repositorio al inicio; las secciones 3 a 10 recogen los cambios posteriores.

Índice

- 1. Estado del proyecto al inicio
- 2. Cómo estaba implementado el ACO al inicio
- 3. ACO: retorno al depósito y comparabilidad con OR-Tools
- 4. Primera comparación ACO frente a GLS y diagnóstico de lentitud
- 5. Mejoras de la búsqueda local y de la construcción
- 6. Esquemas de feromona y número de semillas
- 7. Ablación de la feromona y valor de alfa
- 8. Nuevos operadores, reinicio de feromona y perturbación (ILS)
- 9. Defecto de recarga en OR-Tools y comparación final
- 10. Herramientas nuevas: ACO independiente y ranking de Friedman
- 11. Preguntas conceptuales respondidas
- 12. Archivos modificados o creados, y pendientes

1. Estado del proyecto al inicio

Lo que sigue sale de leer los archivos del repositorio (no se pudo ejecutar git diff; el historial de git pull muestra dos actualizaciones recientes, `95dce6e → 02bc751 → 6eb3c5d`, sin detalle de archivos).

Solver con OR-Tools (run.py, problem/execute/evrp.py)

- Modela el EVRP con tres dimensiones: distancia como costo de arco, **Capacity** (carga de mercancía) y **Fuel** (batería).
- Los clientes son obligatorios (se descartan solo con una penalización alta, `drop_penalty = max(10 000 000, 100·dist_max)`). Las estaciones de carga son nodos opcionales con costo 0; solo en ellas el SlackVar de Fuel puede ser mayor que 0 (energía recargada).
- Acepta `vehicle_states` (el vehículo arranca con posición, batería y carga parciales) y `data_override` (una matriz medida en vivo por SUMO). Son la base del EVRP dinámico.
- Distancias disponibles: Euclidean, Manhattan, Haversine, Chebyshev y OSRM (servicio /table, que además devuelve la matriz de tiempos).

Consumo de batería según el tipo de distancia

No se encontró código que lo seleccione automáticamente: es un parámetro manual. `vehicle_speed` se reutiliza como `fuel_consumption_rate` (energía por km). El comentario de `run.py` indica «consumo: 6 para HAVERSINE y 3 para OSRM», con batería 100. Al inicio, `run_aco.py` usaba 1.0 con OSRM, por lo que ACO y OR-Tools no se comparaban con el mismo consumo (se corrigió, sección 3).

Simulación SUMO (carpeta simulation/)

- Fases 1 a 4: red vial real (OSM a SUMO), rutas del EVRP sobre la red, tráfico de fondo y Battery Device real de SUMO.
- Orquestador dinámico (`dynamic_replanning.py`): corre SUMO hasta un checkpoint, lee el estado real de cada vehículo, fuerza congestión y mide el antes/después, re-optimiza con OR-Tools usando la matriz viva de SUMO y reinyecta las rutas.
- **Más escenarios:** `simulation/run.py` mapea ahora cada instancia a su propia red (Quebec para 40c y 60c, Montreal para 30c, Trois-Rivières para 50c y 90c) y guarda cada escenario en `sumo_scenario/<instancia>/<algoritmo>/`, lo que corrige un error de mezclar rutas de distintos algoritmos.
- Observación: el pull deshizo una edición anterior; la opción 5 de `simulation/run.py` ya no pasa `--gui`.

Limitaciones documentadas en el README

- Cada estación de carga es un único nodo en OR-Tools, así que se puede visitar una sola vez por solución.
- La red no tiene semáforos y el tráfico de fondo aleatorio no genera congestión detectable a esa escala.

2. Cómo estaba implementado el ACO al inicio

Archivo `p_mh/ACO/evrp_aco.py`: solver independiente y funcional (sin clases), que usa el mismo cargador de instancias y escribe la solución en el mismo formato de texto que OR-Tools, de modo que `simulation/` puede consumirla igual. Cada iteración hacía:

- **Construcción** (Thymianis et al., CEC 2022): cada hormiga construye vehículo por vehículo. Clientes y estaciones alcanzables compiten en la misma ruleta con pesos $\tau^{\alpha} \cdot \eta^{\beta} \cdot h^{\gamma}$, donde τ es la feromona, $\eta = 1/\text{distancia}$ y h es un término de energía (una estación pesa en proporción a la urgencia cap/batería − 1; un cliente pesa 1). Las estaciones solo entran a la ruleta si la batería está bajo el 15 % y el vehículo no acaba de recargar.
- **Búsqueda local:** 2-opt dentro de cada ruta, solo sobre la mejor hormiga de la iteración (Liu et al., IEEE Access 2022).
- **Feromona Rank-AS** (Nie et al., SMC 2022): evaporación y depósito de las $w-1$ mejores hormigas ponderadas por rango, más un refuerzo a la mejor global.
- **Costo:** distancia total más penalización por cliente descartado.

Se detectaron dos problemas: (1) si el vehículo no podía volver al depósito y tampoco alcanzaba una estación, igual volvía y la batería quedaba negativa, sin penalización ni marca de infactibilidad; (2) ACO y OR-Tools no eran comparables (consumo distinto, aritmética en coma flotante frente a enteros, formato de salida distinto).

3. ACO: retorno al depósito y comparabilidad con OR-Tools

Retorno al depósito

- Un vehículo solo avanza a un nodo si desde allí todavía puede volver al depósito, directo o pasando por una estación desde la que llegue con batería llena. Una tabla precalculada (`finish_need`) hace la comprobación instantánea.
- El rescate en estación solo se hace si, tras recargar, queda algún cliente que se pueda servir (antes podía ir de estación en estación sin avanzar).
- Las estaciones no tienen límite de visitas en ACO. Cargar sin necesidad se penaliza: cada parada de carga que se podría quitar sin dejar la ruta sin batería suma una penalización (la distancia media de un arco) al costo interno de la hormiga, y la búsqueda local elimina esas paradas si la distancia no empeora.

Comparabilidad con OR-Tools

- Consumo por defecto según la distancia: 3.0 con OSRM, 6.0 con Haversine; `run_aco.py` usa 3.0.
- Misma aritmética entera: costo de arco = int(dist) y consumo de arco = int(dist·rate), igual que `evrp.py`.
- Mismo objetivo (distancia más drop_penalty), misma política de recarga (a batería llena) y mismo formato de salida: Time of the route, Total recharges, Minimum battery (SoC) reached, rutas vacías para vehículos no usados, y la batería de una estación se reporta al llegar (antes de recargar).

4. Primera comparación ACO frente a GLS y diagnóstico de lentitud

Entorno de pruebas: el sandbox no puede instalar OR-Tools ni llegar al OSRM de Docker del usuario, así que ACO se ejecutó con las matrices OSRM guardadas en `instances_data/osrm_cache`, y GLS se tomó de las soluciones guardadas en el repositorio (10 s, batería 100, consumo 3.0). Un validador independiente recalcula la batería arco a arco desde la matriz y comprueba capacidad y que cada cliente se sirva una vez.

Resultado inicial (3 semillas, 10 s, mejor de ACO):

Instancia	GLS	ACO (mejor)	Gap
30c	92	93	+1,1 % (media 95,7)
40c	73	75	+2,7 %
50c	137	141	+2,9 %
60c	131	138	+5,3 %
90c	158	223	+41 %

Perfil de tiempo (90c, 101 nodos): construir una hormiga tardaba ~16 ms (una iteración de 30 hormigas ≈ 0,5 s), la búsqueda local ~8 ms y solo se aplicaba a 1 de cada 30 hormigas. Una hormiga sin refinar daba ~396 km y 2-opt la bajaba a ~337, frente a 158 de GLS: el problema principal era la **calidad de lo construido** y la falta de iteraciones para que la feromona aprendiera (9 iteraciones en 10 s en la instancia grande).

5. Mejoras de la búsqueda local y de la construcción

- **Búsqueda local entre rutas:** además del 2-opt, relocate, Or-opt (segmentos de 1 a 3 clientes, en ambos sentidos) y swap, buscando solo cerca de los vecinos de cada extremo. Si un movimiento rompe la batería se repara insertando una estación; se acepta solo si la distancia real baja. Una hormiga en 90c pasó de 436 a 218 km (antes, 337 con solo 2-opt).
- **Listas de candidatos:** la construcción evalúa solo los 20 clientes factibles más cercanos (entre 1,7 y 4 veces más rápida: 1,5 ms contra 5,8 ms en 50c).
- La búsqueda local se aplica a las 3 mejores hormigas de cada iteración (`local_search_top`).

Resultado (3 semillas, 10 s):

Instancia	GLS	ACO medio	ACO mejor	Gap del mejor
30c	92	94,3	93	+1,1 %
40c	73	74,0	73	0 %
50c	137	142,0	141	+2,9 %
60c	131	138,3	133	+1,5 %
90c	158	168,3	159	+0,6 %

6. Esquemas de feromona y número de semillas

Se compararon cuatro variantes (3 semillas, 10 s, 5 instancias): Rank-AS, Rank-AS con regla ACS (q0 = 0,5), MAX-MIN y MAX-MIN con ACS.

Variante	Gap medio frente a GLS
Rank-AS (base)	+3,2 %
Rank-AS + ACS (q0 = 0,5)	+3,3 %
MAX-MIN	**+2,5 %**
MAX-MIN + ACS	+3,1 %

- **ACS no aporta nada** (reduce la exploración y la búsqueda local ya hace ese trabajo): se eliminó del código por decisión del usuario.
- **MAX-MIN** quedó como esquema por defecto (Rank-AS sigue disponible con `pheromone_scheme='rank_as'`). La diferencia es pequeña y entra en el ruido de 3 semillas; después todos los experimentos se hicieron con 6 semillas.
- **Multiprocessing:** implementado como opción apagada (`workers=1`). Solo paraleliza la búsqueda local (casi todo el tiempo). Con 2 núcleos dio entre +7 % y +29 % de iteraciones. Se dejó en 1 proceso para que la comparación con GLS (un solo hilo) sea justa; en Windows exige el bloque `if __name__ == "__main__"`.

Qué es una semilla: ACO usa números aleatorios, así que dos corridas con los mismos datos dan resultados distintos. La semilla fija esa secuencia aleatoria: con la misma semilla el resultado se repite. Se corre con varias semillas para medir la variabilidad. GLS casi no depende del azar, por eso basta una corrida.

Búsqueda local a más hormigas (6 semillas, 10 s): aplicarla a las 3, 10 o 30 mejores hormigas.

Variante	Gap medio frente a GLS
Top 3	2,48 %
Top 10	2,36 %
Top 30 (todas)	4,74 %

Más búsqueda local por iteración no compensa con el reloj fijo en 10 s y Python: top 10 empata con top 3 dentro del ruido, y top 30 empeora porque hace menos iteraciones. Las 30 hormigas sí cuentan siempre para la feromona; la búsqueda local solo se aplica a las mejores por coste, no por regla del algoritmo.

7. Ablación de la feromona y valor de alfa

Para comprobar que ACO no se reducía a «búsquedas locales aleatorias», se probó α = 0 (desactiva el efecto de la feromona) frente a otros valores (6 semillas, 10 s, media de las semillas):

Instancia	GLS	α=0	α=0,5	α=1	α=2	α=3
30c	92	93,7	93,7	**93,3**	96,8	98,2
40c	73	74,5	73,8	**73,2**	73,3	73,5
50c	137	140,8	141,0	140,2	140,2	**140,0**
60c	131	138,0	136,7	135,7	133,2	**130,7**
90c	158	173,2	170,0	165,7	**160,5**	160,8
Gap medio		4,3 %	3,6 %	2,5 %	2,25 %	2,2 %

- **La feromona sí aporta**: con α = 0, ACO queda en +4,3 % sobre GLS; con α entre 1 y 3, en +2,2 a +2,5 %. Comparado por pares (30 combinaciones instancia × semilla), α = 0 pierde frente a α = 1 en 16 casos y gana en 8.
- Patrón por tamaño: en instancias chicas (30c, 40c) un α alto empeora (la feromona se satura al haber 100 a 150 iteraciones); en grandes (60c, 90c) un α alto mejora mucho (solo hay 20 a 40 iteraciones y hace falta que guíe pronto).
- Se dejó α automático (`alpha=None`): 1,0 si la instancia tiene 50 nodos o menos, y 2,0 si tiene más. Es una regla empírica con 5 instancias; conviene validarla con otras.

Diagnóstico: ¿tiempo o diversificación? (60 s, curva de convergencia)

Instancia (GLS a 10 s)	10 s	30 s	60 s	Última mejora	Lectura
50c (137)	139,5	139,0	139,0	5 a 13 s	estancamiento
60c (131)	133,7	131,0	129,0	hasta 46 s	limitado por tiempo
90c (158)	162,0	157,7	157,7	4 a 22 s	mixto

Se matizó una idea: una población no explora automáticamente más espacio que GLS. OR-Tools es C++ y evalúa millones de movimientos por segundo; este ACO es Python puro y en 10 s construye unas 1000 a 4000 soluciones.

8. Nuevos operadores, reinicio de feromona y perturbación (ILS)

- **Relocate dentro de la misma ruta:** mueve segmentos de 1 a 3 clientes a otra posición de su ruta (900 a 4200 mejoras aceptadas por corrida).
- **2-opt*:** intercambia las colas de dos rutas, cortando para que la nueva arista una a un vecino cercano (190 a 610 mejoras por corrida).
- **Reinicio de feromona** tras N iteraciones sin mejora del mejor del ciclo (el mejor global se conserva).

Instancia	GLS	Antes (α auto)	+ operadores	+ operadores + reinicio
30c	92	93,3 / 92	**92,0 / 92**	92,0 / 92
40c	73	73,2 / 73	**73,0 / 73**	73,2 / 73
50c	137	140,2 / 137	**137,0 / 135**	138,3 / 137
60c	131	133,2 / 130	**131,2 / 128**	131,7 / 128
90c	158	160,5 / 157	**159,8 / 156**	162,3 / 159
Gap medio		1,45 %	**0,26 %**	0,89 %

(Cada celda: media / mejor de 6 semillas.) Los operadores nuevos redujeron el gap medio de 1,45 % a 0,26 %. El reinicio con N = 10 empeora (sobre todo en 90c), por lo que quedó desactivado por defecto (`restart_patience=0`). Los cuatro operadores principales se usan todos en cada corrida.

Perturbación sobre la mejor solución (ILS)

En cada iteración se destruye una parte de la mejor solución global (una ruta entera al azar, o un cliente al azar y sus m vecinos más cercanos), se reinsertan los clientes en la posición más barata que respete capacidad y batería (pudiendo abrir una ruta nueva si hay vehículos libres) y se aplica la búsqueda local. Se acepta si no empeora. Por defecto `ils_per_iter=2`.

Instancia	GLS	Mejor ACO sin ILS	Mejor ACO con ILS	Valores con ILS (6 semillas)
30c	92	92	92	92 en las 6
40c	73	73	73	73 en las 6
50c	137*	135	135	135 en las 6
60c	132	126	126	127, 131, 126, 126, 126, 126
90c	158*	156	**152**	154, 152, 162, 156, 154, 157

La perturbación solo cambia el mejor valor en 90c (156 a 152), pero hace los resultados mucho más regulares: 50c da 135 en las 6 semillas y 60c da 126 en 4 de 6. (*GLS guardado antes de corregir el defecto de recarga; ver sección 9.)

9. Defecto de recarga en OR-Tools y comparación final

Hallazgo. Al validar con batería estricta, 2 de las 5 soluciones GLS guardadas (50c y 90c) resultaron infactibles (batería −4, −9 y −1). En `add_fuel_constraints` la restricción *CumulVar ≤ capacidad* solo acota el nivel al **llegar** a cada nodo, no al **salir** de la estación: OR-Tools podía recargar por encima de 100 y «llegar» al siguiente nodo con 100 aunque el tramo consumiera energía (p. ej. *53[CS] Bat(85) → 20 Bat(100)*). Probablemente explica también la observación de la tutora sobre que las baterías finales no coinciden con 100 − distancia.

Corrección aplicada en problem/execute/evrp.py: en cada estación, batería de llegada + recarga ≤ capacidad; en el depósito de salida no se puede recargar (si el inicio es una estación en re-planeación, tampoco puede superar la capacidad).

Comparación ejecutada por el usuario (6 semillas, 10 s, mejor de ACO frente a GLS)

Primera ejecución (GLS aún con el defecto en 50c y 90c):

Instancia	GLS	GLS factible	Mejor ACO
30c	92	sí	92
40c	73	sí	73
50c	137	**NO**	135
60c	132	sí	128
90c	158	**NO**	156

Segunda ejecución, con la corrección en OR-Tools y con perturbación en ACO:

Instancia	GLS (corregido)	Recargas GLS	Mejor ACO	Semilla	Recargas ACO	Diferencia
30c	92	1	92	1	1	0
40c	73	4	73	1	0	0
50c	**140**	4	135	1	3	−5 (−3,6 %)
60c	130	6	126	3	0	−4 (−3,1 %)
90c	**161**	8	152	2	2	−9 (−5,6 %)

Con GLS ya factible, 50c sube de 137 a 140 y 90c de 158 a 161. El mejor ACO es igual o mejor que GLS en las cinco instancias, y recarga menos (0 recargas en 40c y 60c).

Media y variabilidad de ACO antes de añadir la perturbación (6 semillas, 10 s, entorno de pruebas)

Instancia	GLS guardado	ACO media	Desv.	Mejor	Peor	ACO < = > GLS
30c	92	92,0	0,0	92	92	0 / 6 / 0
40c	73	73,0	0,0	73	73	0 / 6 / 0
50c	137	137,0	1,0	135	138	1 / 3 / 2
60c	131	130,8	2,5	126	133	2 / 0 / 4
90c	158	159,8	2,4	156	164	1 / 0 / 5

En media, ACO y GLS estaban prácticamente empatados en ese momento (+0,28 % en total; en las 30 corridas ACO fue mejor 4 veces, igual 15 y peor 11). El usuario aclaró que le interesa comparar solo el **mejor ACO contra el mejor GLS** (y luego averiguar de qué semilla provino), por lo que `compare_aco_vs_ortools.py` ahora imprime solo esa tabla.

10. Herramientas nuevas

ACO independiente: run_aco.py

```
python run_aco.py                                    # 5 instancias, 6 semillas, 10 s
python run_aco.py --instance quebec_50c_5ev_5cs      # una instancia, 6 semillas
python run_aco.py --instance quebec_50c_5ev_5cs --seed 3
python run_aco.py --seeds 1 2 3 --time 20
```

Guarda cada (instancia, semilla) en `problem/osrm/solutions_evrp_<1000+semilla>/solutions_ACO/` y deja la mejor de cada instancia en `problem/osrm/solutions_evrp_best/solutions_ACO/`. Los parámetros están agrupados en `ACO_PARAMS`.

Ranking de Friedman: friedman_ranking.py

- **Diseño:** bloques = las 5 instancias; tratamientos = cada combinación heurística + metaheurística de OR-Tools y ACO; medida = objetivo (menor es mejor). Rangos con empates promediados y estadístico de Friedman con corrección por empates (implementado sin scipy y verificado con valores conocidos de la chi-cuadrado).
- **ACO entra con una sola semilla** sorteada entre las corridas completas (`--aco-seed random`, o fija con `--aco-seed 3`; sorteo repetible con `--pick-seed`), la misma en todas las instancias, igual que cada combinación de OR-Tools es una corrida.
- **Se excluye lo infactible:** una combinación participa solo si tiene las 5 instancias y todas factibles con batería estricta y sin clientes descartados; si no, sale entera del ranking (Friedman necesita bloques completos) y el script lista cada exclusión con su motivo.
- **Holm** contra ACO es opcional (`--holm`): un extra posterior al ranking.
- **Estado:** el script se probó con datos reales, pero aún no hay ranking: casi todas las combinaciones tenían solo 1 o 2 instancias (resultados viejos de Haversine con otro consumo). El usuario está ejecutando `run.py` con todas las combinaciones (OSRM, batería 100, consumo 3.0, 10 s, ~40 minutos); después se corre `python run_aco.py` y `python friedman_ranking.py`.

Reservas: con 5 bloques y ~50 algoritmos el test tiene poca potencia (un p alto no significa «ACO es igual», solo que no se puede afirmar diferencia); más instancias (10 a 15) lo harían más concluyente.

Híbrido ACO + heurística (pendiente)

Decisión del usuario: ACO se mezclaría solo con heurísticas constructivas (p. ej. PATH_CHEAPEST_ARC), no con metaheurísticas como GLS o Tabu. La idea es sembrar el ACO con una solución inicial de OR-Tools (mejor inicial y refuerzo de feromona), descontando del presupuesto de 10 s lo que tarde en calcularse. Queda anotado para cuando termine el ranking.

11. Preguntas conceptuales respondidas

¿Qué son las feromonas?

Una matriz τ[i][j] que mide qué tan buena ha resultado la arista «ir de i a j» en las soluciones anteriores. Cada hormiga elige el siguiente nodo con probabilidad proporcional a $\tau^{\alpha} \cdot (1/\text{distancia})^{\beta} \cdot \text{energía}^{\gamma}$. Tras cada iteración τ se evapora y la mejor hormiga refuerza las aristas de su solución; es la memoria colectiva del algoritmo. MAX-MIN la acota entre un mínimo y un máximo para evitar estancamiento.

¿Por qué ACO lleva búsqueda local dentro?

Las hormigas construyen paso a paso sin poder corregir lo ya elegido (una hormiga sola en 90c daba 436 km y la búsqueda local la bajaba a 218). En la literatura de VRP el ACO casi siempre va híbrido con búsqueda local, y así la feromona aprende de soluciones ya buenas. Lo que lo distingue de lanzar muchos inicios aleatorios con búsqueda local es la feromona; la ablación de la sección 7 comprobó que aporta.

¿Cómo elegir hormigas, semillas y cuántas mejores se refinan sin prueba y error?

- **Parámetros:** configuradores automáticos como irace (F-Race) prueban configuraciones sobre instancias de entrenamiento (distintas de las de evaluación) y descartan las malas con tests estadísticos.
- **Número de semillas:** $n \approx (1{,}96 \cdot \sigma/E)^2$, con σ la desviación típica observada y E el margen de error deseado (σ ≈ 3 y E ≈ 1 da cerca de 35). En la práctica se usan 10 a 30 y se comparan variantes con las mismas semillas y un test de Wilcoxon.
- **Cuántas mejores hormigas se refinan:** depende del presupuesto de tiempo; se limita para que el tiempo de búsqueda local por iteración sea comparable al de construir.

¿Se puede añadir otra cuenta de Claude Pro como colaboradora?

El uso compartido de proyectos con invitados está disponible solo en planes Team o Enterprise (permisos «Can view» / «Can edit»; los chats siguen privados salvo que se compartan). Una sesión local de Cowork no aparece en esa función. Alternativas: compartir un enlace de solo lectura si existe el botón Share, dejar el contexto en el repositorio (este documento), o pasar la transcripción.

12. Archivos modificados o creados, y pendientes

Archivo	Cambio
p_mh/ACO/evrp_aco.py	Reescrito: retorno al depósito garantizado, aritmética entera como OR-Tools, formato de salida igual, listas de candidatos, búsqueda local entre rutas (relocate, Or-opt, swap, 2-opt*, relocate intra-ruta, 2-opt), poda de cargas innecesarias, MAX-MIN (y Rank-AS opcional), α automático, perturbación ILS, reinicio opcional, multiproceso opcional, registro de progreso
run_aco.py	Ahora es un programa con opciones (--instance, --seed, --seeds, --time); guarda la mejor de cada instancia
compare_aco_vs_ortools.py	Nuevo: corre ACO y GLS con los mismos parámetros y muestra solo mejor ACO frente a mejor GLS, con la semilla de origen y la factibilidad estricta
friedman_ranking.py	Nuevo: ranking de Friedman (ACO con una semilla sorteada; excluye combinaciones infactibles o incompletas; Holm opcional)
problem/execute/evrp.py	Corrección del defecto de recarga (batería al salir ≤ capacidad)
docker-compose.osrm.yml, simulation/*	Cambios de sesiones anteriores (OSRM max-table-size, visualización en SUMO-GUI, tráfico de fondo)
problem/osrm/solutions_evrp_99/, __pycache__	Residuos de pruebas en el repositorio que no se pudieron borrar desde el sandbox: se pueden eliminar a mano

Pendientes

- Terminar la corrida completa de `run.py` (todas las combinaciones, OSRM, consumo 3.0, 10 s), correr `run_aco.py` y luego `friedman_ranking.py`.
- Probar el híbrido ACO + heurística constructiva (sembrado) y comparar contra ACO puro.
- Validar la regla de α automático con más instancias; considerar generar instancias adicionales para dar más potencia al test de Friedman.
- Decidir si se refactoriza el ACO para ajustarlo a los diagramas de patrones de diseño del informe (Strategy + Template Method), cuestión abierta de sesiones anteriores.
- Resolver el error RemoteDisconnected de OSRM si reaparece (se necesita docker ps y docker logs evrp_osrm inmediatamente después del fallo).